

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto: GS Stock & Facturación.

Link del documento de software: [Ir al documento](#)

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Presentación: [Ir a presentacion](#)

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

## Product Backlog

Historias de usuario:

→ [Ir a historias de usuario](#)

## Lista de tareas:

### Estructura y Gestión de Usuarios

- Configurar servidor Express con Node.js.  
Se crea el servidor backend utilizando el framework Express sobre Node.js  
Importancia:
- Configurar Vue.js y estructura del frontend.  
Se establece el entorno de desarrollo con Vue.js y se organiza la estructura de carpetas y componentes  
Importancia: Alta
- Configurar contenedor Docker del proyecto  
Se crean archivos Docker para contenerizar el backend, frontend y base de datos  
Importancia: Alta
- Configurar entorno de pruebas  
Se prepara un entorno para ejecutar pruebas unitarias o de integración  
Importancia: Alta
- Implementar autenticación con JWT  
Se utiliza JSON Web Tokens para autenticar a los usuarios una vez iniciada su sesión  
Importancia: Alta
- Implementar cifrado de contraseñas (bcrypt).  
Se cifra la contraseña de los usuarios antes de guardarla en la base de datos  
Importancia: Alta
- Definir middleware para roles y permisos.  
Se crea lógica intermedia que verifica qué usuarios pueden acceder a ciertas rutas según su rol  
Importancia: Alta
- Crear endpoints de login y logout.  
Se crean las rutas para iniciar y cerrar sesión, validando credenciales y tokens  
Importancia: Alta
- Implementar sistema de recuperación de contraseña.  
Permite a los usuarios restablecer su contraseña mediante correo o token de recuperación  
Importancia: Alta
- Implementar CRUD de usuarios.  
Se implementan las funciones para crear, leer, actualizar y eliminar usuarios  
Importancia: Alta
- Asignar roles y permisos dinámicamente.  
Se permite modificar roles y permisos a los usuarios desde la interfaz o backend  
Importancia: Alta
- Crear UI para gestión de usuarios.  
Interfaz visual para que administradores gestionen los usuarios y sus roles  
Importancia: Alta
- Creación de lógica para cambiar tokens con logout

Se invalida el token actual al cerrar sesión para evitar usos posteriores

Importancia: Alta

→ Implementar frontend de la creación de usuarios

Se desarrolla el formulario y lógica en Vue.js para registrar nuevos usuarios

Importancia: Alta

→ Crear los endpoints de envío de datos de creación de usuario a la base

Se crean rutas que reciben datos del frontend y los guardan en la base de datos.

Se crean rutas que reciben datos del frontend y los guardan en la base de datos

Importancia: Alta

→ Creación de base de datos dentro de docker

Se crea una instancia de base de datos dentro de un contenedor Docker

Importancia: Alta

## **Inventario**

→ Implementar CRUD de productos.

Desarrollar las operaciones básicas (Crear, Leer, Actualizar, Eliminar) para la gestión de productos en el sistema.

Importancia: Alta

→ Implementar historial de movimientos en inventario.

Registrar cambios en el inventario (entradas, salidas, ajustes) para auditoría y trazabilidad.

Importancia: Media-Alta

→ Crear UI para gestión de inventario.

Diseñar y desarrollar una interfaz intuitiva para que los usuarios administren el inventario fácilmente.

Importancia: Alta

→ Implementar WebSockets para actualización en tiempo real.

Usar WebSockets para reflejar cambios en la UI sin necesidad de recargar la página

Importancia: Media

→ Actualizar stock automáticamente tras ventas y pedidos.

Asegurar que el inventario se ajuste automáticamente al confirmar ventas o recibir pedidos.

Importancia: Alta

→ Implementar sincronización en tiempo real con la base de datos.

Garantizar que los cambios se reflejen inmediatamente en la BD y viceversa

Importancia: Media-Alta

## **Facturación**

→ Implementar generación de facturas.

Desarrollar la funcionalidad para crear facturas automáticamente con los datos de ventas/pedidos

Importancia: Media-Alta

→ Implementar envío de facturas por email.

Integrar un servicio de correo para enviar facturas automáticamente a clientes tras su generación.

Importancia: leve

→ Implementar historial de facturación.

Almacenar y organizar un registro de todas las facturas emitidas

Importancia: Media

→ Crear UI para facturación.

Diseñar una interfaz que permita visualizar, gestionar y emitir facturas de manera intuitiva.

Importancia: Media

## **Finanzas**

→ Implementar seguimiento de cuentas por cobrar.

Desarrollar un módulo para monitorear el estado de las facturas pendientes de pago

Importancia: Alta

→ Implementar registro de cobros realizados.

Registrar pagos recibidos (efectivo, transferencias, etc.), asociándolos a facturas específicas y actualizando el saldo pendiente.

Importancia: Alta

→ Crear reportes financieros.

Generar informes automatizados.

Importancia media-alta

## **Gestión de Clientes**

→ Implementar CRUD de clientes.

Desarrollar funcionalidades básicas (Crear, Leer, Actualizar, Eliminar) para la gestión de clientes en el sistema

Importancia: Alta

→ Implementar clasificación de clientes (A, B, C, D).

D Asignar categorías a clientes según prioridad de los clientes

Importancia: Alta

→ Almacenar datos de los clientes.

Definir y estructurar la base de datos para guardar información relevante de clientes

Importancia: Alta

→ Crear UI para gestión de clientes.

Diseñar una interfaz intuitiva que permita registrar, buscar y editar información de clientes eficientemente

Importancia: Alta

## **Reportes y Dashboards**

→ Implementar reportes de ventas.

Desarrollar informes que muestren datos clave de ventas

Importancia: Alta

- Implementar reportes de inventario.  
Generar reportes de stock actual, movimientos, productos más vendidos/ociosos, y alertas de niveles mínimos  
Importancia: Alta
- Implementar reportes financieros.  
Crear informes de ingresos, egresos, cuentas por cobrar/pagar, y rentabilidad  
Importancia: Alta
- Creación a PDF de reportes.  
Permitir exportar cualquier reporte en formato PDF  
Importancia: Media
- Implementar descarga de reportes.  
Habilitar descarga de reportes en múltiples formatos según necesidad del usuario  
Importancia: Media
- Crear UI para visualización de reportes.  
Diseñar una interfaz con filtros dinámicos, gráficos interactivos y opciones de personalización  
Importancia: Alta

## **Pedidos y Descuentos**

- Implementación de pedidos.  
Desarrollar la lógica y estructura para gestionar pedidos  
Importancia: Alta
- Implementación de descuentos.  
Agregar funcionalidad para aplicar descuentos  
Importancia: Media-Alta
- Crear UI para creación de pedidos.  
Diseñar una interfaz intuitiva que permita seleccionar, definir y visualizar los pedidos  
Importancia: Alta

## **Pila del Sprint**

### **Listar las historias de usuario que se elaborarán en el sprint:**

1. Creación de lógica para cambiar tokens con logout
2. Implementar frontend de la creación de usuarios
3. Crear los endpoints de envío de datos de creación de usuario a la base
4. Implementar cifrado de contraseñas (bcrypt) en la creación de usuarios
5. Middleware
6. Configurar contenedor Docker del proyecto

7. Configurar servidor Express con Node.js
8. Configurar Vue.js y estructura del frontend
9. Implementar autenticación con JWT
10. Implementar cifrado de contraseñas (bcrypt)
11. Definir middleware para roles y permisos en login
12. Crear endpoints de login
13. Creación de base de datos dentro de docker
14. Implementar frontend del login
15. Creación de endpoint de logout

**Puntos de historia según horas.**

| Historia de Usuario                                                    | Puntos de Historia | Justificación                                                               |
|------------------------------------------------------------------------|--------------------|-----------------------------------------------------------------------------|
| Creación de lógica para cambiar tokens con logout                      | 5                  | Involucra manejar tokens activos, blacklist y coordinación con login/logout |
| Implementar frontend de la creación de usuarios                        | 5                  | Formulario básico con validación, visual, no muy complejo                   |
| Crear los endpoints de envío de datos de creación de usuario a la base | 5                  | Incluye validaciones, conexión a base de datos, control de errores          |
| Implementar cifrado de contraseñas (bcrypt)                            | 3                  | Técnica conocida, pero requiere pruebas y configuración                     |
| Middleware                                                             | 8                  | Incluye lógica de autorización, roles, etc.                                 |
| Configurar contenedor Docker del proyecto                              | 8                  | Involucra networking, dependencias y ambiente                               |
| Configurar servidor Express con Node.js                                | 5                  | Tarea de inicio del backend                                                 |
| Configurar Vue.js y estructura del frontend                            | 5                  | Crear rutas, vistas, componentes, diseño base                               |

|                                                   |   |                                                          |
|---------------------------------------------------|---|----------------------------------------------------------|
| Implementar autenticación con JWT                 | 5 | Manejo de tokens, expiración, integración backend        |
| Definir middleware para roles y permisos en login | 5 | Control de acceso, requiere lógica condicional y pruebas |
| Crear endpoints de login                          | 3 | Recibe credenciales, valida y responde con token         |
| Creación de base de datos dentro de docker        | 5 | Crear imagen, volumen, conexión desde app                |
| Implementar frontend del login                    | 3 | Form simple, validaciones básicas                        |
| Creación de endpoint de logout                    | 3 | Invalida token o lo revoca                               |

### Tareas para la pila del sprint 1

- Configurar Vue.js y estructura del frontend
- Configurar contenedor Docker del proyecto
- Creación de base de datos dentro de docker
- Implementar cifrado de contraseñas (bcrypt)
- Implementar autenticación con JWT
- Crear endpoints de login
- Crear interfaz del login

### Calendario de planificación del Sprint 1

| Tarea                                       | Fecha de finalización |
|---------------------------------------------|-----------------------|
| Configurar Vue.js y estructura del frontend | 03/04/2025            |
| Configurar contenedor Docker del proyecto   | 03/04/2025            |
| Creación de base de datos dentro de docker  | 03/04/2025            |
| Crear interfaz del login                    | 04/04/2025            |
| Implementar cifrado de contraseñas (bcrypt) | 05/04/2025            |
| Implementar autenticación con JWT           | 05/04/2025            |
| Crear endpoints de login                    | 06/04/2025            |

## **Infraestructura**

### **Elección de la Herramienta de Contenerización**

Para la contenerización de este proyecto, se decidió utilizar Docker como herramienta principal debido a varios factores clave. En primer lugar, el equipo ya cuenta con un nivel de familiaridad considerable con su arquitectura, comandos y flujo de trabajo, lo cual permite acelerar el desarrollo y evitar una curva de aprendizaje innecesaria.

Además, Docker ofrece una solución ligera y eficiente para crear entornos aislados, lo que facilita la configuración, despliegue y mantenimiento de los distintos componentes del proyecto. Su amplio soporte en la comunidad, la abundante documentación y la facilidad para integrarse con otras herramientas del ecosistema lo convierten en una opción sólida frente a otras tecnologías.

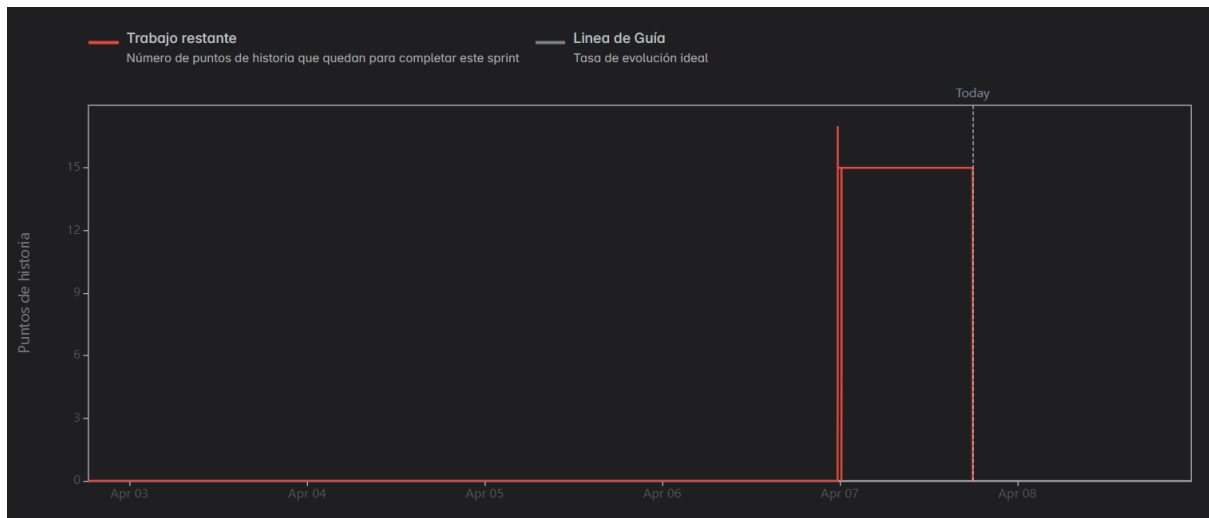
### **Lista de funcionalidades planificadas que se terminaron completamente.**

1. GSS-6: Configurar contenedor Docker del proyecto
2. GSS-7: Configurar servidor Express con Node.js
3. GSS-8: Configurar Vue.js y estructura del frontend
4. GSS-9: Implementar autenticación con JWT
5. GSS-10: Implementar cifrado de contraseñas (bcrypt)
6. GSS-11: Definir middleware para roles y permisos en login
7. GSS-13: Implementar frontend del login
8. GSS-18: Creación de endpoint de logout
9. GSS-19: Creación de lógica para cambiar tokens con logout
10. GSS-20: Creación de base de datos dentro de Docker

## **Resultados del Sprint**

### **Gráficos burndown**

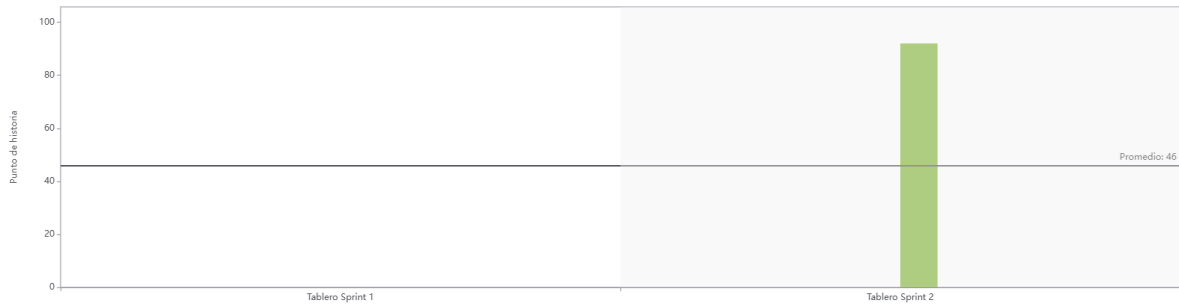




La gráfica burndown del Sprint 1 muestra que se logró completar todas las tareas planificadas dentro del tiempo establecido, lo cual representa un buen resultado en términos de cumplimiento de objetivos. Sin embargo, también revela oportunidades de mejora importantes para el equipo. A lo largo del sprint, el manejo del tiempo no fue el más adecuado, lo que llevó a que varias tareas se ejecutaran de forma apresurada. Esta situación podría comprometer la calidad del producto si no se corrige en los siguientes ciclos.

Otro aspecto relevante que se puede observar en la gráfica es la incorporación de nuevas tareas durante el desarrollo del sprint. Estas correspondieron, en su mayoría, a subtareas no contempladas inicialmente, las cuales surgieron de manera natural conforme se avanzaba en la implementación.

## Gráficos de velocidad



La gráfica de velocidad de Jira muestra que, durante el Sprint, el equipo alcanzó un rendimiento significativamente alto en términos de puntos de historia completados. Esto sugiere que el equipo fue capaz de cumplir con las tareas.

### **Indicador numérico del éxito del sprint junto justificación.**

El sprint actual puede considerarse un éxito, con una nota de 8.5/10, ya que se completaron las tareas más críticas del desarrollo inicial. La infraestructura base está totalmente configurada, el Docker, Express y Vue.js, así como los componentes esenciales de seguridad, que son JWT, bcrypt y middleware de roles. Además, se terminaron tanto los endpoints fundamentales, siendo estos el login y logout, como el frontend básico del login, junto con la documentación inicial de historias de usuario y tareas.

Quedan algunos elementos menores en progreso, como la lógica avanzada de logout y el frontend de creación de usuarios, que representan el 10% pendiente. Las tareas no iniciadas, que son el 5%, son principalmente complementos de seguridad y documentación que no bloquean el funcionamiento básico del sistema.

### **Discusión del éxito del sprint basado en las métricas.**

El Sprint 1 ha sido un éxito en términos generales, con el equipo alcanzando un rendimiento considerablemente alto, completando la mayoría de las tareas clave dentro del tiempo estipulado. La gráfica de velocidad muestra un cumplimiento sobresaliente de las tareas en el backlog, especialmente en las funcionalidades críticas como la configuración del servidor Express, la implementación de JWT para autenticación y el cifrado de contraseñas con bcrypt. Estas tareas, de alta prioridad, fueron completadas a tiempo y con calidad, lo que refuerza la base del sistema.

Sin embargo, a pesar del éxito, la gráfica burndown indica que el manejo del tiempo no fue completamente eficiente. Algunas tareas se ejecutaron apresuradamente, lo que sugiere que, aunque el equipo completó las tareas planificadas, la distribución y organización del trabajo pueden mejorarse. Esto es especialmente relevante para tareas como la lógica de logout y la creación de usuarios desde el frontend, las cuales quedaron parcialmente implementadas. La inclusión de tareas adicionales durante el sprint también contribuyó a esta ligera desorganización, pero estas subtareas emergentes fueron gestionadas de manera que no impidieron el avance general.

### **Evidencias de muestra al product owner.**



En general, al Product Owner le agradaron mucho los resultados obtenidos durante este primer sprint. Se mostró contento al ver que ya existen avances visuales concretos en el proyecto, lo cual generó entusiasmo por el progreso logrado hasta ahora. Durante la revisión, solicitó realizar algunos ajustes en el logo, específicamente para alinearlos con el logotipo oficial de la empresa. Fuera de eso, consideró que la funcionalidad del login está bien implementada y cumple con lo esperado.

Además, aprovechamos la ocasión para explicarle cómo funcionará la metodología de trabajo basada en sprints. Se le informó que, cada cierto tiempo, se llevarán a cabo reuniones programadas para presentarle los avances más recientes. Estas sesiones servirán para obtener su retroalimentación o aprobación, y así definir si debemos hacer correcciones o continuar con las siguientes tareas planificadas.

## **Retrospectiva del sprint.**

¿Qué salió bien?

Se logró completar la infraestructura básica del sistema, configurando Docker, el servidor backend con Express y la estructura del frontend con Vue.js. También se implementaron con éxito componentes importantes de seguridad como JWT para la autenticación, bcrypt para el cifrado de contraseñas y middleware para el control de roles y permisos, lo que fortalece la base del sistema desde el inicio. Además, se desarrollaron los primeros endpoints y el frontend básico del login, lo cual permite una interacción inicial funcional. Por último, la documentación de historias de usuario y la planificación del sprint fueron claras con lo que se desea lograr, lo que facilitó la organización del trabajo.

¿Qué se podría mejorar?

Algunas tareas complejas, como la lógica para el cierre de sesión y la asignación de roles, se pudieron haber dividido en subtareas más pequeñas y específicas para facilitar su desarrollo y seguimiento. También se identificó que el desarrollo del frontend podría beneficiarse de una mejor distribución entre los compañeros para avanzar más rápidamente en la interfaz de usuario.

¿Qué no salió tan bien?

Aunque todas las tareas fueron completadas, la organización y distribución de tareas entre los miembros del equipo podría haberse optimizado para lograr una mayor eficiencia y balance en la carga de trabajo, evitando que algunas áreas del proyecto tuvieran mayor concentración de tareas que otras.